

IILM UNIVERSITY

Industrial Internship Report

on

PASSWORD MANAGER

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Technology in Computer Science and Engineering

Submitted By

Aaditya Narayan

Roll Number: 2410030971

Section: 3CSE23

Batch: 2024-28

Internship Domain: Python Programming

Internship Duration: 5 Weeks

Internship Period: 07 August 2026 - 09 September 2026

In collaboration with

Upskill Campus, The IoT Academy and UniConverge Technologies (UCT)

CANDIDATE'S DECLARATION

I, Aaditya Narayan, hereby declare that this internship report titled "Password Manager" is my own original work, prepared to fulfil the academic requirements of the Bachelor of Technology in Computer Science and Engineering programme at IILM University, Greater Noida. It documents the work and learning I completed during my Python Programming internship. I take full responsibility for the accuracy and originality of the content presented here.

Signature

Student Name: Aaditya Narayan

Roll No.: 2410030971

ACKNOWLEDGEMENT

I would like to thank IILM University, Upskill Campus, The IoT Academy and UniConverge Technologies for the opportunity to complete this internship and for the structured learning path it provided. The weekly modules, mentor sessions and project assignments gave me a practical grounding in Python that I could not have gained from classroom study alone.

I am also grateful to my mentors and faculty for their guidance, and to my friends and family for their encouragement throughout the programme. Building this project week by week taught me as much about persistence and debugging discipline as it did about Python itself.

Signature

Student Name: Aaditya Narayan

Roll No.: 2410030971

TABLE OF CONTENTS

1. Candidate's Declaration.....	i
2. Acknowledgement.....	ii
3. Internship Completion Certificate.....	1
4. Project Description.....	2
4.1 Introduction.....	2
4.2 Organization / Internship Profile.....	2
4.3 Problem Statement.....	2
4.4 Project Objectives.....	2
4.5 Scope of the Project.....	3
4.6 Technologies and Tools Used.....	3
4.7 System Architecture.....	3
4.8 Methodology.....	4
4.9 Achieved Outcomes.....	4
4.10 Project Implementation.....	5
4.11 GitHub Project Submission.....	5
4.12 Security Consideration.....	6
5. Testing and Results.....	6
6. My Learnings.....	7
7. Future Work Scope.....	8
8. Bibliography / References.....	9

3. INTERNSHIP COMPLETION CERTIFICATE

The Internship Completion Certificate issued to Aaditya Narayan is reproduced below as evidence of successful completion. The certificate records the domain as Python and the internship period as 07-08-2026 to 09-09-2026, and identifies the programme as a collaboration between AICTE, UniConverge Technologies and upSkill Campus.



Fig. 3.1: Internship Completion Certificate

4. PROJECT DESCRIPTION

4.1 Introduction

As part of the Python Programming internship, I built a command-line Password Manager — a small utility that lets a user store website credentials in one local file instead of noting them down separately for every service. The project was chosen because it touches most of the core building blocks taught during the internship: functions, control flow, dictionaries, file I/O and JSON serialization, all combined into one working tool rather than isolated exercises.

The current version keeps things deliberately simple. On launch, the program shows a menu with three choices — add a new credential, view the credentials already saved, or exit — and everything is persisted to a local file called passwords.json between runs.

4.2 Organization / Internship Profile

This internship was delivered through Upskill Campus, in partnership with The IoT Academy and UniConverge Technologies (UCT), under an AICTE-recognised structure. The programme combined weekly Python learning modules with short assessments, and concluded with an independent project that had to be built, tested and published on GitHub.

4.3 Problem Statement

People typically use dozens of online accounts, and keeping track of each login manually — in notebooks, sticky notes or scattered text files — is disorganised and error-prone. This project addresses that in a minimal way: a program that takes in a website's credentials, stores them in one structured local file, and lets the user pull them back up through a simple menu, without needing a spreadsheet or a third-party service.

4.4 Project Objectives

- Build a working Password Manager using core Python.
- Apply functions, conditionals, loops and file handling in a single cohesive program.
- Persist credentials in a structured, human-readable JSON format.
- Provide add and view operations through a clear command-line menu.
- Practice testing, debugging and publishing the finished code to GitHub.

4.5 Scope of the Project

This version covers local credential entry, JSON-based storage and retrieval via the command line — nothing more. It is an educational prototype, not a production tool: there is no encryption, no master-password login, and no cloud sync. Those gaps are intentional at this stage and are addressed in the Future Work section rather than being built into this release.

4.6 Technologies and Tools Used

Component	Technology / Tool	Purpose
Programming Language	Python	Application logic and user interaction
Data Format	JSON	Local structured storage
Standard Libraries	json, os	Serialization and file-existence checks

IDE	Visual Studio Code	Writing, running and debugging the code
Version Control	GitHub	Hosting the source for submission

Table 4.1: Technology and Tool Stack

4.7 System Architecture

The program follows a straightforward flow: the user picks a menu option, the corresponding function runs, that function reads from or writes to passwords.json, and a result is printed back to the console. Keeping each operation in its own function was a deliberate choice — it made the program easier to test piece by piece and easier to extend later without touching unrelated code.

4.8 Methodology

Step 1 — Requirement Understanding: Defined the core operations a minimal password manager needs — add and view — before writing any code.

Step 2 — Design: Split the program into small functions: loading stored data, saving data, adding a new entry, and viewing existing entries.

Step 3 — Implementation: Used a Python dictionary for in-memory storage and the json module to persist it to disk.

Step 4 — User Interaction: Wrapped the whole thing in a loop with a text menu, using conditionals to route each choice to the right function.

Step 5 — Testing: Ran the program with both valid and deliberately invalid inputs to check it behaved correctly and didn't crash.

Step 6 — Version Control: Pushed the finished code to a public GitHub repository for submission.

4.9 Achieved Outcomes

The finished program meets its original goals: it starts up and shows the menu correctly, accepts a website, username and password for a new entry, writes that entry to passwords.json, and lists saved records back on request. It also handles an invalid menu choice gracefully — printing a message and re-showing the menu instead of crashing.

4.10 Project Implementation

Storage is handled by two small functions. `load_passwords()` checks whether passwords.json already exists and, if so, loads its contents into memory; otherwise it starts with an empty dictionary. `save_passwords(data)` writes the current dictionary back to the file in a readable, indented JSON format. `add_password()` collects the website, username and password from the user and inserts them into the dictionary before saving, and `view_passwords()` reads back the stored records and prints them to the console.

The main loop presents three options — Add Password, View Passwords, Exit — and uses simple if/elif logic to route the user's choice. Any input outside those three options triggers an "invalid choice" message and returns to the menu rather than exiting.

Representative Code Section

```
import json import os FILE = "passwords.json" def load_passwords():
if os.path.exists(FILE):     with open(FILE, "r") as f:
```

```
return json.load(f)    return {} def save_passwords(data):    with
open(FILE, "w") as f:    json.dump(data, f, indent=4)
```

Fig. 4.2: Core JSON storage implementation

4.11 GitHub Project Submission

The completed source code has been pushed to a GitHub repository created for this internship, containing the main implementation file, PasswordManager.py, for review and version tracking.

Repository Link: [add your GitHub repository URL here]

Project Code: [add the direct link to PasswordManager.py here]

4.12 Security Consideration

This is a learning prototype, and it stores credentials as plain text in passwords.json — there is no encryption or master-password protection yet. It should not be used to store real, sensitive passwords in its current form. This limitation is stated plainly here rather than glossed over, so the report reflects exactly what the code does and does not do.

5. TESTING AND RESULTS

Testing was done by running the program directly in the VS Code terminal and walking through each menu option, checking both normal use and edge cases like empty storage and invalid input.

Test ID	Scenario	Input / Action	Expected Result	Status
TC-01	Application start	Run PasswordManager.py	Main menu appears	PASS
TC-02	Add credential	Choose 1, enter details	Credential saved to file	PASS
TC-03	View credentials	Choose 2	Saved credentials listed	PASS
TC-04	Empty storage	Choose 2 before adding data	"No passwords saved" message	PASS
TC-05	Exit	Choose 3	Program terminates cleanly	PASS
TC-06	Invalid option	Enter unsupported value	"Invalid choice" message	PASS

Table 5.1: Functional test cases

5.1 Test Procedure

I first confirmed the menu displayed correctly on launch, then ran through a full add-password flow to check input collection and saving, followed by the view option to confirm retrieval. Finally, I deliberately entered invalid input to make sure the program reported an error instead of crashing.

5.2 Results

Every core scenario passed. The program displayed the correct menu, accepted and stored new credentials, retrieved saved records correctly, and handled invalid input without crashing. For the small dataset used during testing, there was no noticeable lag or performance issue.

5.3 Limitations of Testing

This was functional testing on a small educational build, not a security audit — no claims are made about encryption strength or resistance to attack. Larger datasets, concurrent access, and database-backed storage would all need dedicated testing in a future version.

6. MY LEARNINGS

This internship pushed me past learning Python syntax in isolation and into putting those pieces together into something that actually works end to end. Watching a menu choice turn into a saved file, and that file turn back into a printed list, made the abstract parts of the course — functions, loops, data structures — click in a way that exercises alone hadn't.

6.1 Technical Learning

- Python fundamentals: variables, functions and control flow.
- Building a menu-driven program using loops and conditionals.
- Using dictionaries to hold structured data in memory.
- Reading from and writing to files.
- Serializing and deserializing data with JSON.
- Basic input handling and debugging.
- Creating and managing a GitHub repository for submission.

6.2 Professional Learning

Beyond the code itself, I learned to break a vague requirement ("manage my passwords") into concrete, buildable steps, to test each piece as I went rather than all at once at the end, and to be upfront in documentation about what the project doesn't do yet rather than overselling it.

6.3 Career Growth

This project gave me a genuine, if small, taste of building something from a blank file to a working tool, and it's sharpened my interest in going further with Python — particularly into databases, basic application security and building interfaces beyond the command line. I feel more confident now taking a rough idea and turning it into running code.

7. FUTURE WORK SCOPE

The current build is a working prototype, not a finished product. Here's what I'd tackle next to move it closer to something genuinely usable.

7.1 Security Enhancements

- Encrypt stored credentials rather than keeping them as plain text.
- Add a master password / login step before the menu is accessible.
- Manage encryption keys properly instead of storing them alongside the data.
- Mask password input at the terminal so it isn't shown while typing.

7.2 Functional Enhancements

- Add a random strong-password generator.
- Add search, edit and delete for existing entries.
- Add a password-strength check.
- Add categories, notes and timestamps per entry.
- Add import/export support.

7.3 Engineering Enhancements

- Move from JSON to SQLite (or similar) once the dataset grows.
- Add automated unit tests instead of manual test runs.
- Build a simple GUI or web front end.
- Add proper exception handling and logging.
- Package the app for easier installation.

7.4 Future Validation

Later versions should be tested more rigorously for confidentiality, data integrity, recovery after failure, performance at scale, usability, and correct handling of unexpected input.

8. BIBLIOGRAPHY / REFERENCES

- Python Software Foundation — Python Documentation and Standard Library reference.
- Python json module documentation and usage examples.
- GitHub Documentation — repositories, commits and source hosting.
- Upskill Campus / The IoT Academy internship learning modules and project brief.
- UniConverge Technologies (UCT) internship programme guidance.
- IILM University academic internship report format, used as the structural reference for this report.

DECLARATION OF TECHNICAL ACCURACY

This report describes the Password Manager prototype as it exists in the GitHub repository at the time of writing. Features such as encryption and authentication are listed under Future Work because they are not part of the current codebase.